

ANTHROPIC 官方创业团队调研

Claude Code

创业团队指南

十倍组织如何更快交付：五条 AI 原生运营原则

源自十余家高速增长创业公司的实践

他们像十倍规模的组织一样交付

30%

ClickHouse
更多功能交付

2-3×

Omni
工程生产力

100%

Clay
Bug 分诊自动化

6,000+

Artemis Security
每周 PR

真正的变化不只是“代码写得更快”，而是把产品开发生命周期从头按 **Claude Code** 重构。

01 人人都能交付

02 自动化繁琐工作

03 信任，但要验证

04 为重建而设计

05 原型 → 内测 → 产品化

RULE 01 人人都能交付

最懂问题的人，应该能直接交付第一个可工作的解决方案。

连接工具

不是把律师、销售拉进 IDE，而是把 Claude 带进他们每天使用的系统与数据。

展示原型

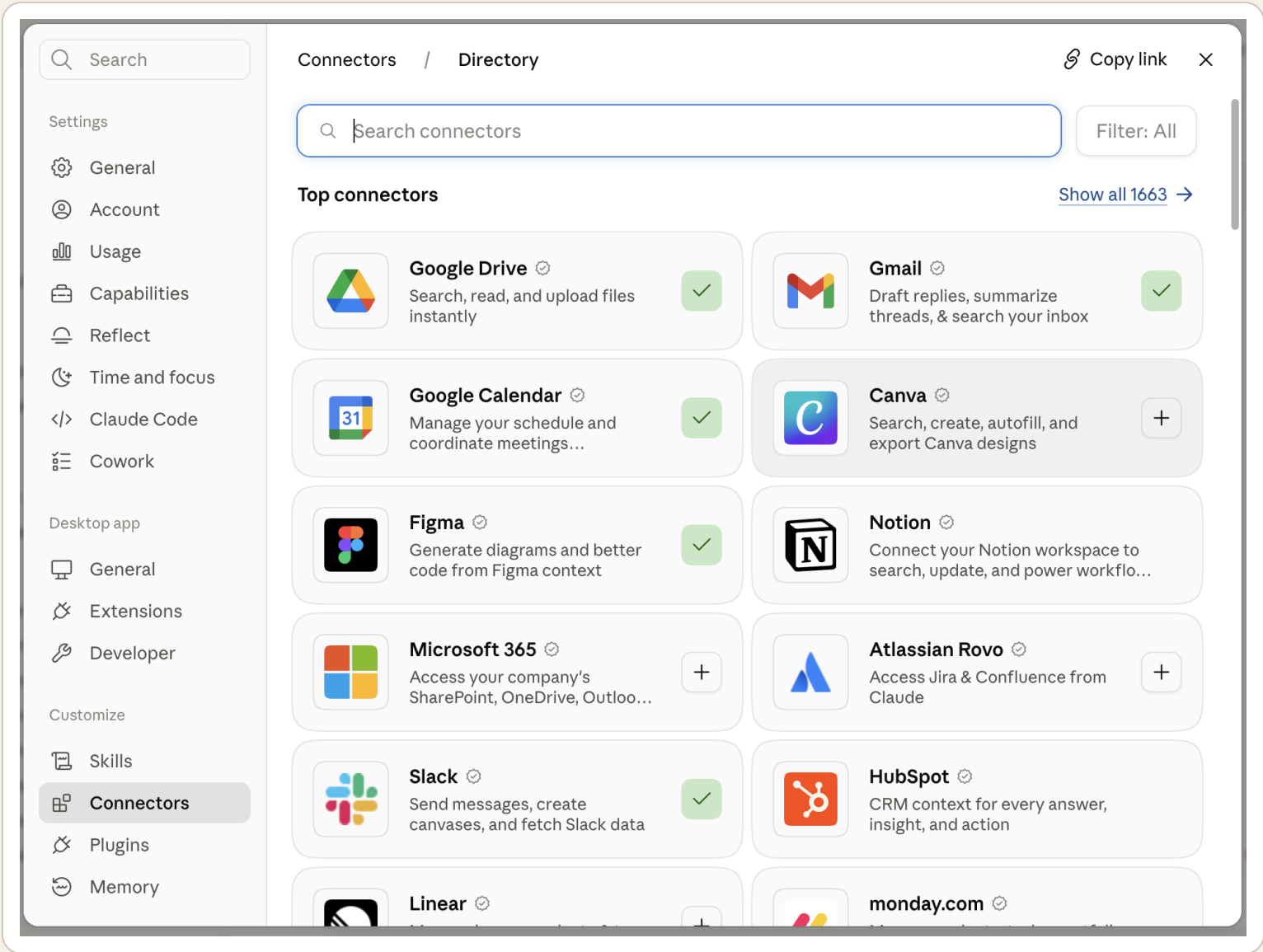
设置季度评审或专用 Slack 频道，让非技术成员的原型进入正式路线图。

共享 Skills

用设计系统、领域知识与流程规范，让“人人交付”仍保持产品一致性。

边界没有消失：非技术成员负责从 0 到 1，设计师和工程师在专业节点接力。

把 Claude Code 带到问题现场



领域上下文

连接业务文件、客户反馈、设计稿与团队沟通。

降低门槛

员工留在熟悉的工具里，也能完成真实工作。

制度化贡献

把偶然的个人尝试，变成可复用的组织能力。

原博客图：Claude 连接器目录，可接入 Google Drive、Gmail、Figma、Notion、Slack、HubSpot 等工作系统。

自动化机械性的 80%

AI 原生 SDLC

入职、写代码、Code Review、测试、CI 与修复，逐步变成闭环。

专用 Agent

数据分析、Bug 分诊、反馈归类、法律文档与医疗数据检查端到端运行。

目的不是替代判断，
而是把人的时间留给真正需要判断的
20%。

Thread #checkout-incidents



Marcus Nguyen 2:14 PM

@Claude checkout error rate jumped from 0.3% to 6% right after the 14:00 deploy — can you dig in and figure out what broke?

1 reply



Claude AGENT 2:14 PM

On it — tracing the checkout error spike since the 14:00 deploy.

- ✓ Pulled the 14:00 deploy diff from GitHub (#4821)
- ✓ Queried Datadog — 502s isolated to the `cart-service`
- ✓ Correlated with a 5x spike in DB connection-pool timeouts
- ✓ Compared error rates by region against rollback candidates
- * Writing root-cause summary + rollback recommendation...

To-dos as of 2:18 PM (Now)

[Open session in Claude](#)

原博客图：在 Slack 事故线程中，Claude 自动拉取 GitHub diff、查询 Datadog、关联数据库超时，并形成回滚建议。

从“辅助写代码”到拥有流程

COMMURE

13 个工单并行

一名工程师编排 Claude subagents，每个智能体独立负责一个 ticket 与 PR。

CLICKHOUSE

自主修复循环

修复 flaky test 与补齐测试覆盖的两个 Agent，已经成为仓库第 2、3 大贡献者。

CLAY

Bug 分诊闭环

从首次分析到建议修复代码，重复流程由 Agent 接手。

先把最频繁、最机械、又能验证结果的流程，做成端到端 Agent。

触发

取上下文

执行

验证

反馈学习

信任，但要验证

自动化的前提不是“模型很聪明”，而是**结果可监控、可审计、可回归**。

- ◆ 根目录 `CLAUDE.md` 写下架构不变量、安全边界与不可妥协项
- ◆ 每次改动跑 golden set 与随机样本，提前发现回归
- ◆ 专家审核模型的推理与修正，所有版本留痕
- ◆ 修复原则，而不是记住单个例子

CAINEX 医疗编码闭环

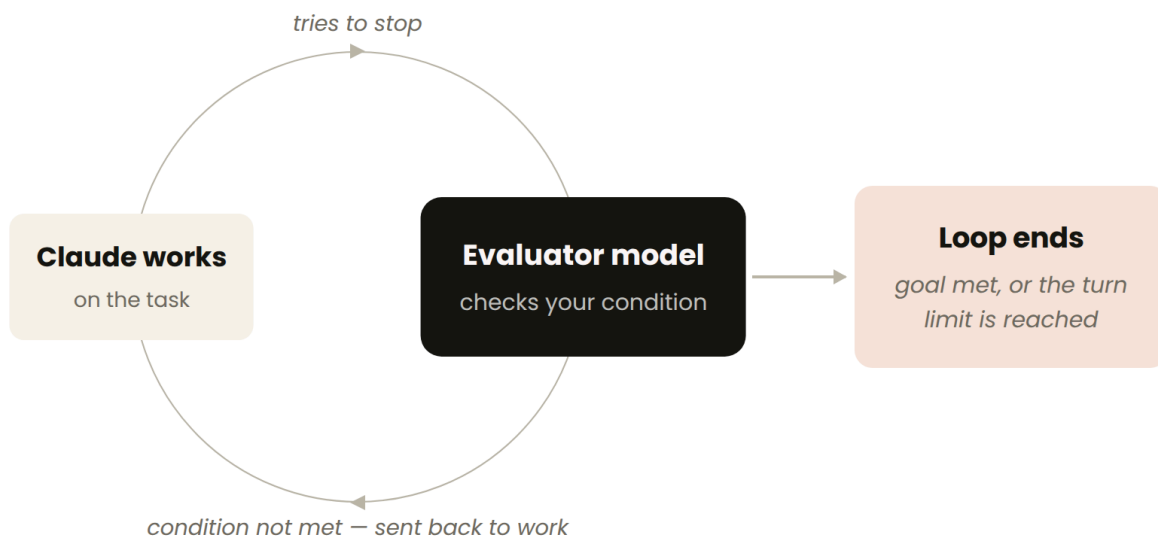
错误不是 typo，而是合规事件

Agent 处理批次 → 审计员检查代码与推理 → Claude 读取纠正 → 修改版本化指令 → 用 golden set 回测 → 只把难例交给工程师。

“Fix the principle, not the example.”

给 Agent 一个可判定的停止条件

/goal get the homepage Lighthouse score to 90 or above, stop after 5 tries



目标

Lighthouse ≥ 90

护栏

最多尝试 5 次

验证者

独立判断是否满足条件

这比“把首页优化一下”更容易形成可靠闭环。

原博客图：Evaluator 检查条件；未达标就把任务送回，达标或达到轮次上限才结束。

RULE 04 为重建而设计

模型能力一直在移动，今天的架构很可能半年后就该推倒重来。

Clay：建四遍

每次重建都带着更清晰的需求，第四次才真正知道系统需要什么。

Commure：拆旧也自动化

用 Skill 扫描已全量发布的 feature flags，并行创建删除旧代码的 PR。

Harvey：敢于 Agent-native

新一波模型能力到来时，完整重构平台，而不是守着沉没成本。

```
~/projects - zsh

$ tree -L 1 ~/projects
~/projects
├── acme-web ← main worktree [main]
├── acme-web-feature ← linked [feature/checkout-flow]
└── acme-web-hotfix ← linked [hotfix/payment-bug]

One repository, one object store – three checkouts you can work in simultaneously, each on its own branch.
```

原博客图：一个仓库、共享对象库，三个独立 worktree 可同时运行 main、feature 与 hotfix。

低成本重建方法：在隔离 worktree 中跑 v2，与 v1 并列评测，只有新版本胜出才合并。

RULE 05

原型 → 内测 → 产品化

用 AI 构建内部 Agent，会反过来提升团队对模型能力、Harness 与产品体验的理解。

01

内部原型

用 Claude Code 快速搭
一个 Agent



02

团队内测

真实工作中持续
dogfood



03

读懂反馈

区分模型问题与 Harness
问题



04

客户产品

成熟后迁移到 API / SDK

Omni

从 Claude Code 的文件式上下文与并行交互获得启发，
简化自己的产品设计。

ClickHouse

用 Claude Code 构建并迭代客户直接使用的 SQL Agent
与 AI SRE。

五条规则组成一个组织飞轮



速度不是“少做检查”，
而是让上下文、执行、验证与学习彼此
连接。

- 更多人能把真实问题变成原型
- Agent 接管机械流程并积累反馈
- 评测与不变量控制风险
- 低成本重建吸收下一代模型能力

创业团队的落地清单

本周就能开始

- ◆ 选择一个高频、机械、结果可验证的流程
- ◆ 连接 Claude 所需的代码、数据与协作工具
- ◆ 在根目录记录架构、安全与业务不变量
- ◆ 给非技术成员一个展示原型的正式入口
- ◆ 把团队最佳实践封装成可共享 Skill

规模化之前

- ◆ 建立并持续更新多个 golden eval sets
- ◆ 重要闭环加入独立验证者与人工审核
- ◆ 用 worktree 隔离并行任务与架构重写
- ◆ 把成功条件、最大轮次与回滚写清楚
- ◆ 先内部 dogfood，再决定是否客户产品化

从一个闭环开始，不要从“全公司自动化”开始。

THE OPERATING SYSTEM

真正的护城河 是自我改进的速度

结构化上下文 · 可验证闭环 · 可重建架构

<https://claude.com/blog/claude-code-guide-for-startups>